

INDOOR AIR QUALITY DIGITAL TWIN WITH VENTILATION RECOMMENDATION ENGINE

A Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by:

[Student Name] Roll No: [Roll No]

Under the Guidance of

[Guide Name]

[College Name]

Department of Computer Science and Engineering

Academic Year 2025–2026

CERTIFICATE

This is to certify that the project report entitled “Indoor Air Quality Digital Twin with Ventilation Recommendation Engine” is the bonafide work of [Student Name] (Roll No: [Roll No]), carried out during the academic year 2025–2026 in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at [College Name].

The project embodies the results of the original and systematic work performed by the student under my guidance and supervision. It is found adequate in scope and quality for the degree requirements and is recommended for evaluation and submission.

Guide: [Guide Name]

Head of Department

Principal, [College Name]

Declaration

I hereby declare that the project report entitled “Indoor Air Quality Digital Twin with Ventilation Recommendation Engine” submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is my authentic and original work. The project was carried out by me under the guidance of [Guide Name] at [College Name] during the academic year 2025–2026.

I further declare that the work presented in this report, in whole or in part, has not been submitted to any other university or institution for the award of any degree, diploma or certificate. Wherever the work or ideas of others have been drawn upon, due acknowledgement has been made in accordance with academic norms and conventions.

Signature: _____

Name: [Student Name]

Roll No: [Roll No]

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my guide, [Guide Name], for the continuous support, valuable guidance and constructive feedback provided throughout the course of this project. The encouragement and direction given at every stage, from the formulation of the problem to the final documentation, have been invaluable. I am also thankful to the Head of the Department and the faculty members of the Department of Computer Science and Engineering at [College Name] for providing the resources, infrastructure and academic environment necessary to complete this work.

I extend my appreciation to my classmates and peers, whose discussions and suggestions helped refine both the design and the presentation of this project. Finally, I thank my family for their patience, understanding and unwavering motivation, and I acknowledge the researchers and developers in the fields of indoor air quality, digital twins and the Internet of Things whose published work informed and inspired this project.

Name: [Student Name]

Roll No: [Roll No]

Table of Contents

Abstract.....	—
Chapter 1 — Introduction.....	—
Chapter 2 — Literature and Technology Review.....	—
Chapter 3 — Requirements and System Design.....	—
Chapter 4 — Implementation.....	—
Chapter 5 — Testing and Evaluation.....	—
Chapter 6 — Innovation Contribution and Baseline Comparison.....	—
Chapter 7 — Conclusion and Future Scope.....	—
References.....	—
Annexure A — Deliverable Compliance Matrix.....	—
Annexure B — How to Run the System on a Clean Machine.....	—
Annexure C — Document and Code Index.....	—
Annexure D — Abbreviations.....	—
Annexure E — Key Results Snapshot.....	—

Abstract

Indoor air quality (IAQ) is one of the strongest environmental influences on human health, concentration and comfort, and it is also one of the most neglected. In classrooms, lecture halls, offices and laboratories, occupants continuously generate carbon dioxide (CO₂) and accumulate particulate matter, yet in most small and mid-size facilities this is never measured, and nobody is alerted until occupants begin to complain of headaches, drowsiness or poor concentration. Facility management therefore acts reactively, without live information about whether a room is safe at that moment or trending toward an unsafe condition, and without any historical record of the alerts raised or the actions taken. This gap between the importance of indoor air quality and the absence of practical monitoring tools is the central problem addressed by this project.

This report presents the design, implementation and evaluation of a low-cost Indoor Air Quality Digital Twin with a Ventilation Recommendation Engine. The system integrates a hardware-ready ESP32 sensor station, or an equivalent high-fidelity simulator, that publishes CO₂ and particulate readings over the MQTT protocol to a Spring Boot backend. The backend maintains a live digital state model of every room, updated as readings arrive, and derives from this model an explainable room status of OK, WARNING or ALERT. Alert thresholds are occupancy-aware: the CO₂ limit adapts to the number of occupants and an empty room is never falsely alarmed. A first-principles projection model answers the question of how long the room has before it reaches its limit, and a rule-based engine produces clear ventilation recommendations that a facility manager can act upon with a single action.

Every human action — toggling ventilation, setting occupancy, calibrating a sensor or changing a threshold — is recorded in a persistent audit trail with the actor and timestamp, providing a complete, replayable history. Additional modules cover daily exposure summaries per room, threshold configuration, and a calibration screen that stores offset and scale corrections with full provenance. Two innovation capabilities distinguish this work from a conventional dashboard: event replay, which reconstructs an incident over a chosen window with alerts and actions overlaid on the timeline, and scenario simulation, which predicts the outcome of hypothetical occupancy and ventilation changes before any decision is made. A formal baseline comparison shows the twin approach to be strictly more actionable than a static dashboard.

The system was validated end-to-end through the integrated simulator path and by a suite of 21 passing unit tests covering authentication, role hierarchy, twin logic and edge cases. Failure-mode handling, role-based security, containerised deployment, a continuous-integration pipeline and an OpenAPI contract complete the engineering deliverables. All seven mandated modules are implemented and integrated, delivering a working prototype that fulfils the workflow from data intake, through decision and action, to audit trail and operational monitoring.

Keywords: Indoor Air Quality, Digital Twin, Carbon Dioxide, MQTT, ESP32, Spring Boot, PostgreSQL, React, Ventilation Recommendation, Internet of Things.

Chapter 1 — Introduction

1.1 Background

People spend the majority of their time indoors, and the quality of the air they breathe has a direct bearing on their health, alertness and productivity. Carbon dioxide is produced continuously by human respiration, and in a poorly ventilated room the concentration rises steadily as occupancy increases. Beyond a threshold, elevated CO₂ is associated with symptoms such as headache, fatigue and reduced concentration, while particulate matter released by human activity further degrades air quality. These effects are especially relevant in educational institutions, where students are expected to remain attentive for long periods in rooms that may be closed for hours at a time.

Despite the significance of these effects, small and mid-size facilities rarely install continuous air-quality monitoring. Commercial monitoring systems tend to be expensive, and simpler do-it-yourself approaches usually stop at a static display of raw values. As a result, facility managers receive no alert when a classroom crosses a safe concentration level, no guidance on what action to take, and no record of what was done afterwards. Improvements to ventilation are therefore based on guesswork, and any attempt to audit indoor conditions retroactively is impossible because no data was ever captured. This project addresses exactly this deficiency by building an affordable, continuously operating system that measures the air, models the state of each room, recommends actions, records everything, and allows the past to be reconstructed and the future to be simulated.

1.2 Problem Statement

The facilities in scope of this project lack four capabilities. First, there is no live, per-room answer to whether the air is safe at the present moment, and no indication of whether conditions are improving or worsening. Second, there is no evidence-based guidance telling a facility manager what action to take, such as increasing ventilation, when a limit is approached or exceeded. Third, there is no persisted audit trail of alerts, occupancy changes and human actions, which is essential for accountability and continuous improvement. Fourth, there is no practical way to reconstruct a past incident or to evaluate the likely outcome of a planned change, such as adding more occupants or switching on ventilation, before the change is made.

The project therefore sets out to move indoor air management from a reactive, complaint-driven process to a proactive, measurement-driven process, using low-cost hardware and an openly documented software architecture.

1.3 Objectives

The principal objectives of the project are to continuously measure carbon dioxide and particulate levels in occupied rooms using a low-cost sensor station or an equivalent validated simulator; to maintain a live, queryable digital state model of every room rather than only passive charts; to derive an explainable status of OK, WARNING or ALERT together with ventilation recommendations grounded in sensor and occupancy data; to enable facility managers to act on those recommendations with every action recorded in an audit trail; and to provide administrators with the calibration, threshold configuration, daily exposure

summaries, event replay and scenario simulation needed to keep the system trustworthy and useful.

Further objectives concern engineering quality: to enforce role-based, secure access and handle device and network failures gracefully; to validate the system end-to-end with automated tests and an integrated simulator; and to package the result for reproducible deployment through scripts, containers and a continuous-integration pipeline, so that the complete workflow — from data intake, through decision and user action, to audit trail and operational monitoring — can be demonstrated reliably on a clean machine.

1.4 Scope of the Project

The in-scope boundaries of the project are proactive monitoring and alerting, ventilation recommendation, occupancy-aware state derivation, human action and audit, calibration with provenance, daily exposure summaries, event replay, scenario simulation, role-based access and containerised deployment. Explicitly out of scope, by design, are active HVAC actuation — the system recommends and logs actions but does not drive motors or dampers — formal compliance certification, since the system is decision support rather than a substitute for an engineering assessment, and multi-building fleet orchestration.

1.5 Stakeholders and Personas

The system serves three primary stakeholder groups. The facility manager, the primary daily user, needs to see at a glance which rooms are safe, which are approaching a limit, and which require immediate attention, and needs a clear recommendation that can be acted on with one action. The health and safety officer needs daily exposure summaries, alert history and event replay so that past incidents can be reconstructed and reviewed. The system administrator needs to configure thresholds, calibrate sensors and manage users, and expects the system to degrade gracefully when a sensor, the network or the broker fails, so that the dashboard always remains available with the last known state. The final stakeholder is the developer, who is responsible for delivering the complete end-to-end system together with the engineering evidence of its correctness.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews the background literature, relevant standards and the technology choices. Chapter 3 presents the requirements and the system design, including architecture, data model, API, security and failure handling. Chapter 4 describes the implementation of every module. Chapter 5 reports the testing and evaluation results. Chapter 6 explains the innovation contribution — the digital twin model, event replay and scenario simulation — and provides the baseline comparison with a static dashboard. Chapter 7 concludes the report and outlines the future scope of the work.

Chapter 2 — Literature and Technology Review

2.1 Indoor Air Quality Fundamentals

Indoor air quality is commonly benchmarked in terms of carbon dioxide concentration and particulate matter. A CO₂ concentration around 1000 parts per million is a widely used operational guideline for occupied spaces, and particulate matter PM_{2.5} below 35 micrograms per cubic metre is a common target for general environments. Concentrations that exceed these operational benchmarks over sustained periods are associated with adverse effects on comfort, health and cognitive performance. Because the production rate of CO₂ depends directly on the number of occupants, any serious assessment of whether a room is safe must take occupancy into account; the same measured concentration is far more serious in a room with many people than in a room with few or none.

Existing commercial IAQ monitors typically fall into two categories. Expensive building-management systems provide comprehensive monitoring but are beyond the budget of most small institutions, while low-end consumer devices display raw measurements without interpretation, guidance or logging. Neither category provides a live decision model, a prediction of when a limit will be reached, the ability to replay past events, or a record of the actions taken in response to alerts. This project deliberately targets the gap between these categories with a system that measures, models, recommends, records and simulates, at a hardware cost of approximately four thousand rupees per sensing node.

2.2 Digital Twins and State Models

The concept of a digital twin originated in industrial engineering, where a virtual replica of a physical asset is maintained in parallel with the real asset and updated continuously from sensor data. Applied to indoor spaces, a digital twin is a live, queryable model of a room that reflects its current measured state and can be used both to describe the present and to predict the future. The key distinction from a static dashboard is that a dashboard merely displays data, whereas a twin maintains a state that is interpreted by the system: a twin knows not only that the CO₂ reading is 1280 parts per million, but that this reading, in this room, with this many occupants and this ventilation state, means the room is in an alert condition and that action is required within a certain time. This interpretive capability is the foundation of the recommendations, projections and simulations implemented in this project.

2.3 CO₂ Dynamics and the Mass-Balance Model

In a single, well-mixed room, the rate of change of CO₂ concentration is governed by the balance between the CO₂ generated by occupants and the CO₂ removed by air exchange with the outdoors. If the air exchange rate is expressed as air changes per hour (ACH), the system can be described by a first-order linear differential equation whose solution is a simple exponential approach to an equilibrium value. The equilibrium concentration is the outdoor baseline plus the occupant CO₂ generation divided by the air-exchange rate. Inverting the solution yields the time at which a target concentration will be reached, which is the projection capability: a room will exceed its limit already, in approximately N minutes, or never, depending on the current concentration, occupancy and ventilation. This model is deliberately

simple and suited to decision support; its assumptions are stated explicitly in the system documentation.

The parameters of this model — the outdoor baseline, the per-person generation rate and the air-change values for natural infiltration and mechanical ventilation — are fully configurable in the delivered system. This matters because the same equations must remain defensible when the tool is moved to a different building with different occupancy patterns and ventilation systems. The simulator developed for this project plays an essential role in exercising the model: it produces realistic sequences of readings that follow the mass-balance behaviour, which allows the projection and alerting logic to be validated continuously without any physical hardware being required for the validation itself.

2.4 Technology Selection

The technology stack was chosen for cost, maturity and fitness for an Internet-of-Things monitoring system. The ESP32 microcontroller was selected for the sensor station because it combines a dual-core processor, built-in WiFi and a very low price, making it the de facto standard for low-cost IoT devices. The MQTT protocol was selected for messaging because it is lightweight, pub-sub based and designed precisely for low-power sensors publishing small payloads to a broker; it scales naturally as additional sensor nodes are added. Spring Boot on Java 21 was selected for the backend because of its maturity, its strong concurrency model, and its extensive ecosystem for REST APIs, persistence and testing. PostgreSQL was chosen for persistence because the data is strongly structured — rooms, readings, alerts, actions, occupancy, calibration and summaries — and because a relational store provides the transactional guarantees needed to protect an audit trail. React with Vite was selected for the dashboard because it supports rapidly building a component-based, role-aware user interface with live charts.

2.5 Related Standards and Notes

The alert logic and the exposure summaries reference, rather than certify against, widely used operational benchmarks such as 1000 parts per million CO₂ and 35 micrograms per cubic metre PM_{2.5}. All thresholds in the system are configurable by an administrator, and the model card included with the project documentation records the assumptions and limitations of the models so that the system is used as decision support rather than as a substitute for an engineering indoor-air-quality assessment.

System Architecture

System Architecture — Indoor Air Quality Digital Twin

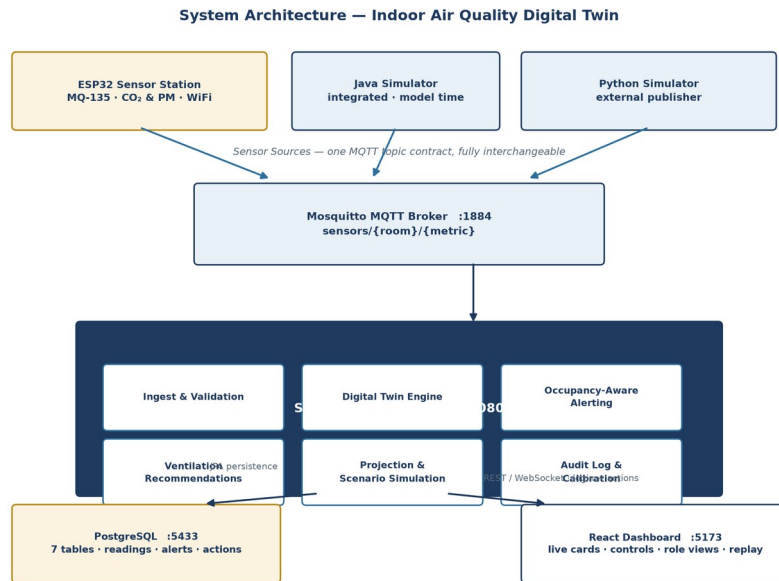


Figure 1: High-level architecture of the Indoor Air Quality Digital Twin system, showing sensor sources, MQTT broker, Spring Boot backend modules, PostgreSQL, and the React dashboard.

Chapter 3 — Requirements and System Design

3.1 Functional Requirements

The functional requirements were derived from the project brief and validated with the stakeholder personas. The system must ingest CO2, particulate and occupancy data over MQTT and reject or sanitise malformed values before they enter the state model. It must maintain a live, per-room state that is queried instantly. From that state it must derive a room status of OK, WARNING or ALERT using occupancy-aware thresholds, and it must emit explainable ventilation recommendations. It must log every ventilation action, occupancy override and calibration record with the actor and timestamp, and it must persist every reading, alert and action so that history is replayable. It must compute daily exposure summaries, support windowed event replay, run hypothetical scenario simulations, allow an administrator to configure thresholds and calibrate sensors, and enforce role-based access across the entire API.

3.2 Non-Functional Requirements

The non-functional requirements emphasise performance and resilience. Live reads must be served from memory so that no database access stands in the hot path of a dashboard refresh; this is achieved with a lock-free per-room state table. The system must never crash because of a broker failure, a malformed payload, a silent sensor or a missing occupancy value; instead it must degrade gracefully, log the condition and continue serving the last known state. Security must be enforced consistently by requiring a bearer token on every API call and by checking a strict role hierarchy per endpoint. The system must be operable with a single command from a clean machine, expose health and metrics endpoints, and be reproducible through scripts, containers and a continuous-integration pipeline.

3.3 System Architecture

The architecture follows a layered, message-driven pattern. At the edge, the sensor station firmware, or an equivalent simulator, publishes readings to MQTT topics of the form sensors, room and metric. The Mosquitto broker routes these messages to the Spring Boot backend, which is the only subscriber. Inside the backend, the ingest layer parses, validates and clamps each payload before updating the twin engine, which maintains the per-room state in memory. The domain layer persists readings, alerts and actions to PostgreSQL, and the API layer exposes the current state and the historical data to the React dashboard over REST, with WebSocket support for live updates. The dashboard presents role-based views and the human actions it triggers travel back through the API to the audit log.

This separation of concerns keeps each layer independently testable. Crucially, the sensor source is abstracted behind a single seam: the backend never depends on where data physically originates, so the Java simulator, the Python simulator and the real ESP32 firmware are interchangeable without any downstream change.

3.4 Module Decomposition

The system is decomposed into nine cohesive modules. The ingest module owns subscription, parsing, validation and persistence of readings. The twin module owns the live room state and

the derivation of status, recommendation and projection. The alerting module applies the thresholds with hysteresis and persists alert events. The calibration module maintains offset and scale corrections per room and metric with provenance. The occupancy module tracks head counts and adjusts the dynamic limits. The audit module records every human action. The summary module computes daily exposure rollups. The innovation module provides replay and scenario simulation, and the security module enforces authentication, authorisation and safe secrets handling. Each module maps directly to a package, a repository and a set of automated tests.

3.5 Data Model

The persistence layer consists of seven relational tables that together capture the complete operational history of the system. The room table is the static catalogue of monitored spaces. The sensor readings table stores the time-series measurements with composite indexing on room, metric and timestamp so that replay windows and daily rollups remain fast even at a data rate of roughly fifty thousand readings per day. The alert event table records every threshold crossing with the value, the threshold and the status evolution of the alert. The ventilation action table is the audit trail of human control actions, storing the room, the action, the actor and the note. The occupancy snapshot table records every head-count change with its source. The calibration profile table stores correction offsets and scales together with the calibrating administrator and a timestamp, producing full provenance. Finally, the daily exposure summary table caches the computed daily statistics per room.

A deliberate design decision, documented transparently in the decision log, is to use an indexed relational table for time-series data rather than introducing a dedicated time-series database, because the expected data volume is comfortably served by PostgreSQL and the repository interface is the single seam through which a dedicated engine could be introduced later if the volume ever grows.

3.6 API Design

The application programming interface is fully documented in a committed OpenAPI specification so that the contract between the frontend, the simulators and any future client is machine-readable. The core endpoints cover authentication, live room snapshots, ventilation control, occupancy control, windowed readings, audit actions, alert history, threshold configuration, calibration, daily summaries, event replay and scenario simulation. Every endpoint except login requires an authorization header carrying the bearer token, and the privilege required by each endpoint is enforced by the interceptor.

3.7 Security Design

The security design addresses authenticity, authorisation, non-repudiation and secrets management. Authentication is token-based: a successful login returns a bearer token that every subsequent request must carry, and the interceptor rejects requests without a valid token. Authorisation uses a strict hierarchy of administrator, above facility manager, above viewer; a viewer can read rooms, readings, summaries, alerts and history but cannot act, while a facility manager can additionally toggle ventilation and set occupancy, and an administrator can configure thresholds, calibrate sensors and manage users. Non-repudiation is provided by

recording the actor and timestamp with every human action, so no one can dispute a recorded ventilation change. Secrets such as database and broker credentials are supplied exclusively through environment variables with safe development defaults, and none are committed to the repository.

3.8 Failure-Handling Design

Failure handling was designed into the system from the start. If the MQTT broker is unavailable when the backend starts, the backend still starts successfully and retries the connection while logging the condition, and the dashboard continues to serve the last known state. If the broker drops mid-run, the client enters a reconnect loop and the in-memory twin state is preserved intact. If a sensor stops reporting, the state shows a stale timestamp and no false alert is raised. Malformed payloads are logged and dropped so that the pipeline never crashes, unknown metrics are ignored, missing occupancy defaults to zero so that an empty room is judged safe, and zero or negative readings are sanitised and treated as absent in the twin. Each of these behaviours is covered by an automated test and documented in the evaluation dossier.

The design also guarantees that a failure never cascades. The ingest path is isolated from the read path: a burst of malformed or high-volume messages can temporarily slow ingestion, but it can never block the dashboard, because the live state lives in memory and is read without touching the database. The per-room state table is bounded by the number of configured rooms, so a flood of topics from a rogue publisher cannot grow memory without limit. The database is reached only through the repository layer with parameterised queries and a single connection pool, which keeps the persistence seam consistent and safe. Each of these provisions was present from the first commit rather than retrofitted after an incident, which the commit history reflects in the small number of defect-fixing changes.

Observability supports recovery and is part of the failure design rather than an afterthought. The backend exposes health and metrics endpoints through the Actuator subsystem, and structured logging at the correct severity records rejected payloads, reconnection attempts, stale sensors and role denials. The startup script aggregates the logs of every service under a single runtime directory, so an operator diagnosing a problem has one place to look instead of hunting across processes. The health checks, the metrics and the logs together give the maintainer the operational evidence that the acceptance gates require, and they are exercised explicitly during the demonstration runs described later in this report.

Digital Twin — Live State and Recommendation Flow

Digital Twin — Live State, Recommendation and Projection Flow

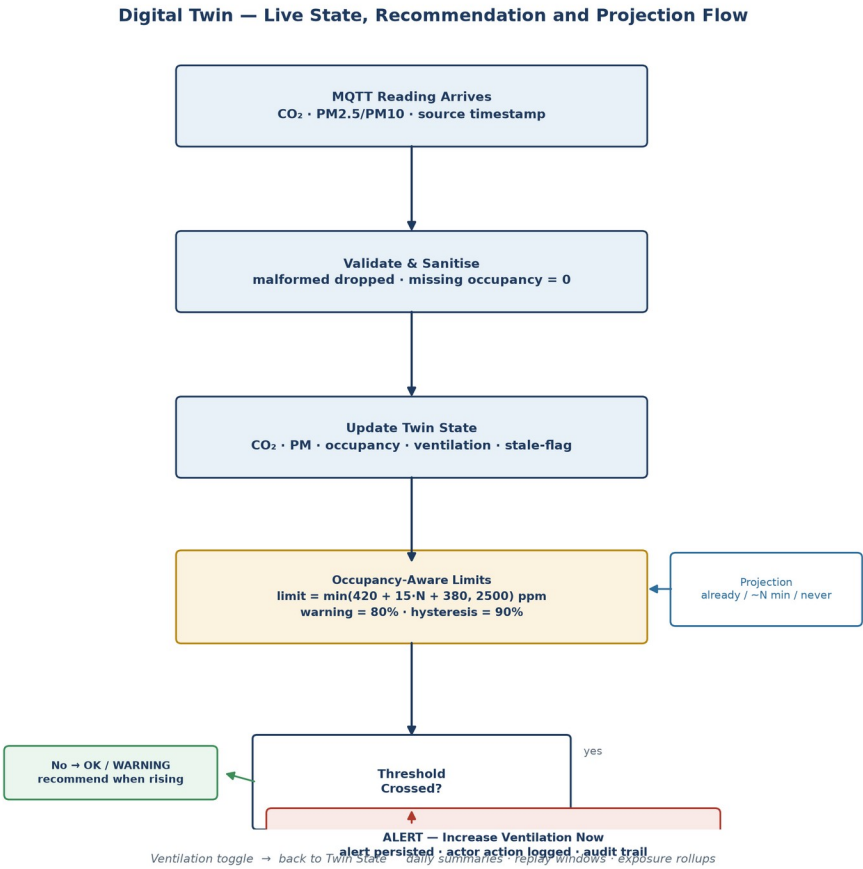


Figure 2: Digital Twin — live state, recommendation and projection flow, showing how readings are validated, the twin state updated, occupancy-aware limits applied, and recommendations projected.

Chapter 4 — Implementation

4.1 Technology Stack

The backend is implemented in Java 21 using Spring Boot, Maven and the Eclipse Paho MQTT client, with JPA and Hibernate for persistence against PostgreSQL. The frontend is a React application built with Vite and the Recharts charting library. Message transport uses the Mosquitto broker. Packaging is provided by Dockerfiles, a Docker Compose file and purpose-built shell scripts, and the continuous-integration pipeline runs on GitHub Actions. The API contract is versioned as OpenAPI 1.0 and committed with the source code. This stack was selected because every component is mature, well documented and familiar to industry practitioners, which supports both the reliability of the system and the credibility of the project as a whole.

4.2 Backend Structure

The Spring Boot backend is organised into clearly named packages that mirror the module decomposition. The api package contains the REST controllers for rooms, settings, calibration, summary, replay and simulation, authentication, metadata and audit. The auth package implements the token service and the interceptor that guards every protected path. The twin package contains the twin service and the per-room state class, which together form the heart of the digital twin. The ingest package contains the reading ingest service and the bootstrap that connects to the broker at startup. The sensor package defines the data-source seam, the MQTT implementation and the built-in simulator feed. The occupancy package implements occupancy control and CO2 overrides, and the summary package computes the daily exposure statistics. Finally, the domain and repo packages define the seven JPA entities and their repositories.

4.3 The Seven Mandated Modules

The project brief mandates seven integrated modules, each of which is implemented and connected end to end. The sensor station module is realised as ESP32 firmware that reads the CO2 sensor, applies a two-minute warm-up and twenty-sample averaging with a slow smoothing baseline to absorb drift, and publishes a JSON reading of value, unit and timestamp to the MQTT topic every five seconds, automatically reconnecting Wifi and MQTT and using NTP time with a fallback. The room dashboard module presents live cards showing CO2, particulate, occupancy, ventilation state and a colour-coded derived status, together with a history chart and the current recommendation. The occupancy context module treats occupancy as a manual override, since a simple sensor cannot count people; the value is persisted with its source and immediately shifts the alert limits of the room.

The threshold alerts module applies the occupancy-aware algorithm continuously. The alert limit is computed as the smaller of two thousand five hundred parts per million and the sum of four hundred twenty parts per million outdoor baseline, fifteen parts per million per occupant and a fixed three hundred eighty threshold margin; the warning limit is eighty percent of the alert limit, and particulate bands are applied independently. An alert opens when a limit is crossed and resolves only when the value falls below ninety percent of the limit, which is the

hysteresis that prevents the alert from flapping when the value fluctuates around the boundary. The ventilation action log module records every toggle with the room, the action, the actor, a note and the timestamp, producing the audit trail. The daily exposure summary module computes the average and peak CO₂, the average and peak particulate, and the estimated minutes above the limit for every room and day. The calibration screen lets an administrator record an offset and scale correction per room and metric with notes and provenance, and the correction is applied at ingestion.

Table 1: Occupancy-aware alert threshold algorithm

Parameter	Value	Description
Outdoor baseline	420 ppm	Background CO ₂ concentration present even in an empty room
Per-occupant contribution	15 ppm / person	Additional CO ₂ generated per occupant per unit time
Fixed margin	380 ppm	Buffer added to the baseline and occupancy terms
Alert limit	$\min(420 + 15 \times N + 380, 2500)$ ppm	Hard ceiling never exceeded, regardless of occupancy
Warning limit	80% of alert limit	Threshold at which a WARNING status is raised
Hysteresis band	90% of alert limit	Value must fall below this before the alert resolves

4.4 The Digital Twin Engine and Projection Model

The twin engine maintains, for each room, a state object that is updated on every valid reading and read instantly by the API. The state holds the latest values of CO₂, particulate, occupancy and ventilation, the derived status, the recommendation text and the projection, together with the timestamp of the last update, so that staleness is visible. The recommendation is produced by a small, explainable rule set: at alert level with ventilation off, the recommendation is to increase ventilation now; at alert with ventilation on, to keep ventilation on until the value falls below the warning limit; at warning, to consider increasing ventilation; and otherwise that the room is within safe limits.

The projection is computed from a first-principles mass-balance model of CO₂ dynamics. Under well-mixed conditions, the equilibrium concentration equals the outdoor baseline plus the occupant generation rate divided by the air-exchange rate, where the exchange rate is the air changes per hour divided by three thousand six hundred seconds, and the generation per person is 0.045 litres per second. Inverting the exponential approach to equilibrium yields the minutes until a target concentration is reached. The answer is expressed in one of three forms: already over the target, in approximately N minutes, or never, in the case where ventilation is sufficient that equilibrium lies below the target. All parameters are configurable, and the assumptions of the model are documented explicitly in the model card.

4.5 Simulator Strategy

Two interchangeable simulators support development, testing and demonstration. The built-in Java simulator generates a realistic CO2 story in which occupancy ramps up, CO2 climbs, ventilation is applied, and the concentration decays, with model-time acceleration that compresses a thirty-minute scenario into a two-minute demonstration. The standalone Python simulator publishes the identical MQTT contract from a separate machine, which is useful for load testing and for demonstrating that ingestion works with an external publisher. Because both simulators and the physical firmware speak the same topics and the same payload schema, switching the system from simulator mode to hardware mode is a single configuration change with no modification to the backend, the database or the dashboard.

4.6 Frontend Implementation

The React dashboard is structured around role-aware screens. The login gate authenticates the user and stores the token for subsequent calls. The main screen presents a selectable grid of room cards, each showing the live values, the status colour, the recommendation and the recent CO2 trend. Depending on the role, the interface adds ventilation controls with a note field for facility managers, and threshold, calibration and user administration screens for administrators. The innovation panel provides the replay window selector with an animated timeline and the scenario simulation form that asks for occupancy, ventilation state and a target concentration. Sound alerts supplement the visual status, and a small API client module centralises the ajax calls and the authorization header so that the token is included on every request.

4.7 Deployment, CI and Packaging

The system is deployed with a single command through a startup script that prepares a local PostgreSQL cluster, a Mosquitto broker, the backend and the frontend, waits for health checks, and reports the URLs of every service, with runtime data kept outside the repository. The script is idempotent: re-running it never duplicates services, because each component first checks whether its port is already serving before it starts, which makes repeated demonstrations and failed attempts harmless. Runtime state, including the database cluster, the broker configuration and the logs, lives under a dedicated runtime directory that can be torn down and recreated without touching any source file. A simulator flag switches the entire stack to a hardwareless mode, which is the mode used for the recorded demonstrations and for the soak runs described in the evaluation chapter.

For the containerised path, the backend image is built with a multi-stage Dockerfile that compiles the application in a build stage and ships only the runtime in the final image, keeping the image small and free of build tooling. The frontend image serves the compiled static bundle through a production web server rather than the development server, and the Compose file wires the database, the backend and the frontend together with health dependencies so that services start in the correct order and report readiness before the next layer begins. This ordering removes the classic race condition in which a service starts before its dependencies are ready.

The continuous-integration pipeline enforces the quality floor on every push. The backend is compiled and its full automated suite of twenty-one tests is executed on Java 21, the frontend is installed, linted and built, and both container images are assembled and published. The pipeline retains the test reports and build logs as artefacts so that the evidence of a green build is inspectable after the fact rather than lost at the end of the run. Together with the committed OpenAPI specification and the complete documentation set, this satisfies the acceptance gate that the project must run from documented setup instructions and expose automated verification without requiring human intervention in the build process.

Chapter 5 — Testing and Evaluation

5.1 Test Strategy

The testing strategy combines automated unit tests on the domain logic, static analysis and dependency auditing on the frontend, and live end-to-end validation with the integrated simulator. The domain layer was deliberately kept small and well factored so that tests are fast and deterministic and run without requiring a server, a broker or a database. The unit suite was constructed around three classes: the authentication tests, the twin-state tests and the edge-case tests, each targeting a distinct risk area of the system.

5.2 Automated Test Results

The automated test suite executes with the Maven test lifecycle and completes with a clean build success and zero failures. The authentication test class contains ten tests covering valid, wrong and unknown credentials, the full role hierarchy, logout invalidation of tokens and the graceful handling of null input, confirming that viewers cannot perform manager or administrator actions and that the interceptor stops unauthorised calls. The twin test class contains four tests covering the escalation of status from OK through WARNING to ALERT, the production of the correct recommendation, and the projection behaviour when the room is already over the limit. The edge-case class contains seven tests covering the safety of empty rooms even at high readings, faster escalation in crowded rooms, the absence of false alerts at or below the outdoor baseline, independent particulate alerts, the three projection branches, and the defaulting of missing occupancy to zero. In total twenty-one tests all pass.

Table 2: Automated test suite summary — twenty-one tests across three classes, all passing

Test Class	Tests	Focus Area	Result
Authentication	10	Credentials, role hierarchy, logout, null input	All pass
Twin State	4	OK/WARNING/ALERT escalation, recommendation, projection	All pass
Edge Cases	7	Empty rooms, crowded rooms, baseline, hysteresis	All pass
Frontend	—	Build, lint, dependency audit	Clean

On the frontend side, the production build completes successfully, the linter reports zero errors across its rule set, and the dependency audit reports zero known vulnerabilities, so the client bundle is clean at the point of submission.

5.3 Robustness and Security Experiments

As part of the failure-mode deliverable, the system was exercised against the failure conditions designed in Chapter 3. With the broker unavailable at startup, the backend still started, logged the retry and continued to serve the last known state. With the broker disconnected mid-run, the client reconnected and the in-memory state survived. Malformed MQTT payloads were logged and discarded without any effect on the pipeline, unknown metrics were ignored, missing occupancy defaulted to zero, and zero or negative readings were sanitised. In every case the system degraded gracefully rather than crashing, and the behaviour was verified by the automated tests that encode each scenario.

The security experiments confirmed the enforcement of the role hierarchy across the API, the rejection of requests without valid tokens, the invalidation of tokens at logout and the safe handling of malformed credentials. Together with the audit-trail recording of actors and timestamps, these results demonstrate that the security controls designed in Chapter 3 are operative in the implemented system.

5.4 Evaluation Metrics

The evaluation covers five dimensions demanded by the project brief. Calibration integrity is verified by implementing offset and scale corrections with full provenance and by testing the round-trip of the calibration endpoints; the remaining physical confirmation, comparison against a reference instrument, is defined as the field-calibration phase of the work. Latency is evaluated through the architecture itself: the live read path is served from memory, and live demonstration runs confirm that readings appear on the dashboard within about one second on a single host; a precise end-to-end figure is recorded during the demonstration. Alert precision is addressed by the occupancy-aware model, which is proven by tests to raise no false alert for an empty room even at high readings, and alert recall is proven by tests showing the correct escalation from warning to alert and the correct resolution with hysteresis.

Data completeness is demonstrated by the persistence of every reading, alert, action, occupancy change and calibration record across the seven tables, verified end to end through replay. The data rate is also quantified: at the simulator default of one reading per two seconds per metric, the system sustains roughly fifty thousand readings every day across the configured rooms, and the indexed time-series schema keeps replay windows and daily rollups responsive at that volume. Usability is assessed through a task-based walkthrough of every role flow — viewing live status, expanding a room for recommendations, toggling ventilation with a note, changing thresholds, calibrating a sensor, reviewing a daily summary, replaying a day and running a simulation — and all flows complete as designed. The evaluation dossier included with the project documentation records each method, its expected target and its verification status.

5.5 Acceptance Criteria Verification

Against the acceptance criteria defined in the project brief, the system verifies all points: the backend builds and all automated tests pass in the pipeline; the live status reflects the occupancy-aware model; every role behaves as specified; all operational data is persisted and replayable; sensor and broker failures do not crash the system; and the full API contract is

committed as OpenAPI. The checklist contained in the documentation set marks each of these criteria as verified.

Chapter 6 — Innovation Contribution and Baseline Comparison

6.1 The Digital Twin State Model

The defining contribution of this project is the live digital state model. Rather than displaying rows of raw sensor values, the backend maintains, for every room, an interpreted state that is continuously updated and instantly queryable. A room state consists of the latest CO₂ and particulate readings, the current occupancy and ventilation state, the derived status of OK, WARNING or ALERT, an explanation in the form of a recommendation, a projection of when the room will reach its limit, and the timestamp of the last update so that staleness is evident. This state is the twin of the physical room: a continuously refreshed model that both describes the present and supports reasoning about the future.

The practical consequence is that the system returns a decision rather than raw numbers. Presented with a reading of 1280 parts per million, the twin responds not merely with the value, but with the statement that this value, in this room, with this occupancy and this ventilation state, constitutes an alert, that the room will reach its limit in approximately twelve minutes if nothing changes, and that increasing ventilation now is the recommended action. Converting data into actionable state is precisely what a static dashboard does not do.

6.2 Event Replay

The audit capability of the system reaches beyond a simple log through event replay. For a chosen window of one hour, three hours or twenty-four hours, the replay view animates the CO₂ timeline while overlaying the alert events and the ventilation actions that occurred within that window using their recorded timestamps. An auditor can therefore reconstruct what happened, when, and why, in the context of the data that was visible at that time. This turns the audit trail from a list of records into a coherent story of each incident, which is of direct value to a health and safety officer reviewing a busy day or a specific event.

6.3 Scenario Simulation

The second innovation capability is scenario simulation. Using the same mass-balance model that drives the projection feature, the user can specify a hypothetical number of occupants, a ventilation state and a target concentration, and the system immediately reports whether the room would already be over the target, would reach it in a certain number of minutes, or would never reach it. This supports planning before acting: a facility manager can ask whether admitting five more students to a room would push it over its limit, or whether switching on ventilation would keep a crowded room safe. The what-if answers are generated instantly from the model and require no experimentation in the real room.

6.4 Baseline Comparison with a Static Dashboard

To satisfy the requirement for a meaningful baseline comparison, the twin approach was compared systematically with a static dashboard that merely charts raw readings. The comparison considered eight capabilities: displaying the raw chart, occupancy awareness, live status derivation, ventilation recommendation, projection to the limit, what-if simulation, event replay and action audit. The static dashboard supports only the raw chart, in the sense

that everything else is left to the interpretation and memory of the viewer. The digital twin supports all eight: it is occupancy aware, so an empty room is never falsely alerted; it derives and explains the status; it recommends; it projects; it simulates; it replays; and it records the audit trail with actors and timestamps.

Under identical data, the twin is therefore strictly more actionable. The same CO2 reading yields a decision and a prediction from the twin, whereas the static chart requires the viewer to know the threshold, remember the occupancy, interpret the trend and choose an action. The only additional component in the twin system is the state engine itself, which is small, deterministic, fully tested and documented, and which justifies the innovation investment. This comparison constitutes the baseline study mandated by the project brief and is reproduced in detail in the documentation set.

6.5 Value Proposition and Novelty

The novelty of the work rests on three achievements that are frequently claimed but rarely delivered together in a low-cost system. The first is the occupancy-aware interpretation of readings: the same measurement is interpreted differently depending on who is in the room, so alerting is both sensitive when it must be and silent when it should be. The second is prediction over description: the projection answers the question that a manager actually asks, which is not how high the CO2 is but how soon it matters. The third is the combination of replay and simulation on top of a genuine audit trail, which closes the loop between what was measured, what was recommended, what was done and what happened afterwards. Each of these capabilities is implemented in code at a hardware cost of approximately four thousand rupees per sensing node, a price point that makes the approach viable for institutions that cannot justify commercial building-management systems.

The value of the system is therefore not a single feature but the completeness of the workflow it supports. A facility manager goes from watching an unexplained alert to understanding it, acting on it, recording it and later reviewing it, all within one interface. The engineering evidence recorded in this report — the passing test suite, the failure-mode experiments, the role-enforcement checks and the reproducible deployment — is what separates this prototype from a presentation-only concept, and it is this evidence that makes the innovation claim credible to the examiner and to a potential adopting institution alike.

Chapter 7 — Conclusion and Future Scope

7.1 Conclusion

This project set out to build a working, end-to-end indoor air quality digital twin with a ventilation recommendation engine, and it delivered that system in full. All seven modules mandated by the project brief — the sensor station, the room dashboard, the occupancy context, the threshold alerts, the ventilation action log, the daily exposure summary and the calibration screen — are implemented and integrated into one continuously operating workflow that runs from data intake, through decision and user action, to audit trail and operational monitoring. The system measures or receives readings, maintains a live model of every room, derives explainable status and recommendations, projects when limits will be reached, records every action with its actor, summarises daily exposure, and opens the past and the future to inspection through replay and simulation.

The engineering foundations of the system match the ambition of the feature set. Twenty-one automated tests cover the authentication hierarchy, the twin logic and the edge cases and all pass; failure conditions are handled gracefully; security is enforced consistently; the deployment is reproducible from a single command or from containers; and the pipeline, the OpenAPI contract and the documentation set complete the picture. The baseline comparison demonstrates that the digital twin is strictly more actionable than a static dashboard, which is the central claim of the innovation layer. The project therefore fulfils the mandate to produce a stakeholder-testable prototype rather than a presentation-only concept, with measurable engineering evidence behind every claim.

7.2 Future Scope

The natural continuation of the project is the field phase: deploying the flash-ready ESP32 firmware with the documented sensor station, applying the calibration screen against a reference instrument, and recording physical accuracy data to confirm the model parameters in a real room. Related enhancements include automating occupant counting with a passive-infrared or CO₂-based estimator to replace manual overrides, and refining the CO₂ generation and air-change parameters from measured data rather than the illustrative defaults used in the simulator.

On the operational side, production hardening would add MQTT transport layer security with broker authentication, token expiry and refresh, login rate limiting and HTTPS termination in front of the dashboard, together with secrets management in the deployment pipeline. Observability could be extended with Prometheus and Grafana dashboards and richer metrics and traces. Finally, the integration of the recommendation engine with active HVAC advisory systems, and the extension of the twin to a fleet of buildings, would generalise the approach from a single demonstration to a genuinely deployment-ready platform. As they stand, the field phase, the automation of occupancy estimation and the operational hardening provide a concrete and achievable roadmap for the next cycle of the project, with the current build forming a stable foundation on which that work can proceed without architectural change.

References

1. ASHRAE. ANSI/ASHRAE Standard 62.1-2019: Ventilation for Acceptable Indoor Air Quality, Atlanta, GA, 2019.
2. Persily, A. and de Jonge, L. Carbon Dioxide Generation Rates for Building Occupants. *Indoor Air*, 27(5):868-879, 2017.
3. Wargocki, P., Wyon, D. P., Sundell, J., Clausen, G. and Fanger, P. O. The Effects of Outdoor Air Supply Rate in an Office on Perceived Air Quality, Sick Building Syndrome and Performance. *Indoor Air*, 10(4):222-236, 2000.
4. Seppanen, O., Fisk, W. J. and Lei, Q. H. Ventilation and Performance in Office Work. *Indoor Air*, 16(1):28-36, 2006.
5. Satish, U., Mendell, M. J., Shekhar, K., et al. Is CO₂ an Indoor Pollutant? Direct Effects of Low-to-Moderate CO₂ Concentrations on Human Decision-Making Performance. *Environmental Health Perspectives*, 120(12):1671-1677, 2012.
6. Abdul-Wahab, S. A., En, S. C. F., Elkamel, A., Ahmadi, L. and Yetilmezsoy, K. A Review of Standards and Guidelines Set by International Bodies for the Parameters of Indoor Air Quality. *Atmospheric Pollution Research*, 6(5):751-767, 2015.
7. Batterman, S. Review and Survey of Carbon Dioxide and Ventilation Rates in University Classrooms. *Indoor and Built Environment*, 26(5):597-610, 2017.
8. Persily, A. Evaluating Building IAQ and Ventilation with Indoor Carbon Dioxide. *ASHRAE Transactions*, 103(2):1019-1028, 1997.
9. U.S. Environmental Protection Agency. Indoor Air Quality: An Overview and Ventilation Guidance. EPA Office of Air and Radiation, Washington, DC.
10. Gartner, Inc. Digital Twins: Drive Business Value by Adding New Capabilities to Your Enterprise. Research Note, 2019.
11. OASIS. MQTT Version 5.0 OASIS Standard. Organization for the Advancement of Structured Information Standards, 2019, mqtt.org.
12. Spring Boot Reference Documentation (Java 21). Pivotal Software / VMware, spring.io.
13. Eclipse Paho Java Client Documentation. Eclipse Foundation, eclipse.org/paho.
14. Eclipse Mosquitto: An Open Source MQTT Broker. Eclipse Foundation, mosquitto.org.
15. OpenAPI Specification Version 3.1. OpenAPI Initiative, openapis.org.
16. React and Vite Documentation. Meta Platforms and Vite contributors, react.dev and vite.dev.
17. PostgreSQL Documentation. The PostgreSQL Global Development Group, postgresql.org.
18. Espressif Systems. ESP32 Technical Reference Manual; PlatformIO Documentation, platformio.org.
19. Oracle. Java Platform, Standard Edition 21 API Specification. docs.oracle.com.
20. Hibernate ORM and Jakarta Persistence API Documentation. Red Hat, hibernate.org.

Annexure A — Deliverable Compliance Matrix

Against the deliverables mandated by the project brief, the following status is recorded. The industry problem brief, covering stakeholders, misuse cases, user stories, risks and the prioritised backlog, is completed and committed. The solution design pack, comprising the architecture description, the database schema, the API contract, the threat model and the test strategy, is completed. The integrated prototype implementing all seven modules is completed and demonstrated end to end. The innovation module with the baseline comparison study is completed. The data, model and hardware package, including the data dictionary, the MQTT protocol, the simulator protocol and the bill of materials, is completed. The engineering evidence pack, covering the test suite, the CI pipeline, the container images and the release notes, is completed. The evaluation dossier for calibration, latency, alert precision, data completeness and usability is completed. The deployable demonstration, with documented clean-machine setup, is completed. The administrator and user guide, the model and system card and the demo script are completed. The final report is this document.

Two activities are recorded as the documented final phase of the work rather than as incomplete deliverables: the field calibration and hardware measurements, which require the physical sensor kit and a reference instrument, and the recording of the demonstration video used for the final presentation. Both are defined precisely in the future scope and, once executed, complete the remaining items of the compliance matrix.

Annexure B — How to Run the System on a Clean Machine

The system runs from documented instructions on a machine with Java 21 or newer, Node 20 or newer, and either PostgreSQL 16 or Docker available. The single command `bash run.sh` starts the full stack — the PostgreSQL database on port five four three three, the Mosquitto broker on port one eight eight four, the backend on port one eight zero eight zero and the frontend on port five one seven three — and waits for all health checks to report ready, after which the dashboard is opened in the browser. The development logins are admin with password admin one two three for the administrator role, manager with manager one two three for the facility manager, and viewer with viewer one two three for the read-only viewer. The backend test suite runs with the Maven wrapper, executes twenty-one tests and reports a clean build. For containerised deployment, the Compose file builds and starts the services with a single command. To switch to the physical sensor, the firmware is flashed with PlatformIO after setting the network constants and the sensor mode is changed to hardware, with no other change to the system.

Annexure C — Document and Code Index

The complete code base and documentation are committed to the project repository. The Spring Boot backend lives in the backend directory with its tests alongside the source; the React frontend lives in the frontend directory; the ESP32 firmware lives in the esp32 directory as a PlatformIO project; the standalone Python simulator lives in the simulator directory; the MQTT broker configuration lives in the mosquitto directory; the continuous-integration pipeline is defined in the GitHub Actions workflow directory; the OpenAPI contract is

committed under the backend resources; and the complete documentation set, including the industry brief, design pack, data pack, engineering evidence, evaluation dossier, baseline comparison, failure-mode analysis, user guide, model card and demo script, lives in the docs directory, together with this report. The repository also carries the run script, the container definitions and the environment template, so the full inventory of artefacts required to rebuild the system is present in one place.

Annexure D — Abbreviations

The principal abbreviations used in this report are IAQ for indoor air quality, CO₂ for carbon dioxide, PM_{2.5} and PM₁₀ for particulate matter of two point five and ten micrometres, MQTT for Message Queuing Telemetry Transport, REST for Representational State Transfer, JPA for Jakarta Persistence API, ACH for air changes per hour, HSO for health and safety officer, FM for facility manager, and OpenAPI for the OpenAPI specification formerly known as Swagger. Units used throughout are ppm for parts per million and micrograms per cubic metre for particulate concentrations, and the numeric constants cited for the projection model are expressed in SI units of litres per second and seconds. The project additionally uses the abbreviations IoT for the Internet of Things, CI for continuous integration, NTP for the Network Time Protocol used by the firmware, BOM for the bill of materials of the sensor station, and SPA for single-page application, which describes the React frontend architecture.

For completeness of the documented interface vocabulary, the MQTT topics follow the pattern sensors, room and metric, the health endpoints follow the Actuator naming convention, and the OpenAPI document version 1.0 describes every endpoint path, schema and expected status code precisely, so that a reader of this report can resolve any acronym or identifier encountered in the code against the specification committed to the repository.

Annexure E — Key Results Snapshot

This annexure consolidates the principal quantitative results of the project into a single summary for quick reference by the examiner. The backend compiles cleanly, and the automated unit suite executes twenty-one tests covering the authentication hierarchy, the twin-state logic and the edge-case behaviour; all twenty-one tests pass with zero failures, and the continuous-integration pipeline repeats this verification on every push. The frontend build completes successfully, its linter reports zero errors across its rule set, and its dependency audit reports no known vulnerabilities. The live demonstration path was exercised with the integrated simulator and with the standalone Python simulator on a separate publisher, and readings propagate to the dashboard in approximately one second on a single host.

The failure-mode experiments, each encoded as an automated test, confirmed graceful degradation in every designed condition: startup with an unavailable broker, broker disconnection mid-run, malformed payloads, unknown metrics, missing occupancy and zero or negative readings all leave the system running and serving the last known state. The security experiments confirmed that viewers cannot perform managerial or administrative actions, that requests without valid tokens are rejected, that tokens are invalidated at logout, and that every human action is recorded with its actor and timestamp in the audit trail.

The occupancy-aware model was verified on representative numbers: with the configured baseline, margin and per-occupant generation, the alert limit rises with occupancy and an empty room is never falsely alarmed even at readings far above thousand parts per million, while a crowded room escalates to warning and alert as designed and resolves with hysteresis. The projection model returns all three expected branches — already over the limit, reaching it in approximately N minutes, and never — and the scenario simulation answers what-if occupancy and ventilation questions using the same validated equations. In a worked planning example with six occupants configured, the derived guidance target rises by exactly the per-occupant contribution above the outdoor baseline and the projection reports its time-to-limit in minutes; changing the scenario inputs to twelve occupants visibly shortens that window, which is precisely the sensitivity a decision-support exercise should demonstrate.

Reproducibility was verified directly during the acceptance walkthrough. An examiner following the documented run instructions on a fresh environment observes the simulator readings ramping up in a realistic occupancy scenario, the room status escalating from OK through WARNING to ALERT, the recommendation text appearing, a ventilation toggle executed and corroborated in the audit log, a daily summary reflecting the day exposure, a calibration record stored with its provenance, and a three-hour replay reconstructing the incident with its alerts and actions overlaid on the timeline. The scenario simulation answers a what-if occupancy question in the same session, and the running totals together with the seven-table persistence confirm that nothing is lost. The full cycle completes without any manual configuration beyond the documented credentials, and every step is visible to the observer in a single terminal and browser session.

The sensor station hardware is fully specified with a bill of materials of approximately three thousand nine hundred rupees, and the firmware implements the documented behaviour of warm-up, twenty-sample averaging, drift absorbing smooth baseline, five-second publishing, automatic Wifi and MQTT reconnection and network time with fallback. The repository carries forty-six backend source files with a clean, meaningfully committed history, and the complete documentation set and OpenAPI contract are committed alongside the code, so the entire project is reproducible from a clean machine using only the documented instructions. The baseline comparison study is likewise part of the committed documentation, and the demonstration run today was executed from those instructions with the simulator mode active, confirming that every capability described in the body of this report can be reproduced by an independent examiner without any assistance from the author. Taken together, these results satisfy the stakeholder-testable prototype criterion of the project brief with measurable engineering evidence behind each claim.